



Today 4:05 PM

Next up: why Server Actions exist and what `"use server"` actually does. 🔥

Lesson 5.1 — Why Server Actions Exist & What `"use server"` Actually Does 🔥

This is a **really important Next.js concept**, because it connects everything we've learned about Server Components with **user interactions**.

Let's start from the problem.

1. The problem: Server Components can't handle browser events

Imagine our analyzer has:

`</>` TypeScript



```
export default async function AnalyzerPage() {
  const data = await getPerformanceData();

  return (
    <button onClick={() => analyzeAgain()}>
      Analyze Again
    </button>
  );
}
```

This looks reasonable, but there's a problem:

Server Components cannot have browser event handlers.

Why?

Because this code:

`</>` TypeScript



```
onClick={() => analyzeAgain()}
```

needs to execute **inside the browser** when the user clicks.

But a Server Component executes on the **server**.

So we normally solve this by creating a Client Component:

```
</> TypeScript

"use client";

export function AnalyzeButton() {
  return (
    <button onClick={() => analyzeAgain()}>
      Analyze Again
    </button>
  );
}
```

Now the browser can handle the click.

But we have another problem.

Where does `analyzeAgain()` run?

Suppose `analyzeAgain()` needs to call Google's PageSpeed API:

```
Browser
  ↓
analyzeAgain()
  ↓
Google PageSpeed API
```

If we put that logic directly into the Client Component, we'd potentially expose things that should remain server-side:

```
</> TypeScript

"use client";

async function analyzeAgain() {
  const response = await fetch(
    `https://pagespeed...?...API_KEY=SECRET`
  );
}
```

😬 **Bad idea.**

The browser shouldn't receive your server secrets.

2. Before Server Actions, we'd commonly create an API endpoint

You could do:



For example:

```
</> TypeScript

"use client";

async function handleClick() {
  await fetch("/api/analyze", {
    method: "POST",
    body: JSON.stringify({
      url: "example.com",
    }),
  });
}
```

And on the server:

```
/api/analyze
  ↓
Google PageSpeed API
  ↓
return result
```

This works.

But Next.js asks:

Why make developers manually create an API endpoint for every server-side mutation?

That's where **Server Actions** come in.

3. Server Actions give the client a way to call server code

Instead of manually designing:

```
Client
  ↓
POST /api/analyze
  ↓
Route Handler
  ↓
server function
```



you can define a server function:

```
</> TypeScript

"use server";

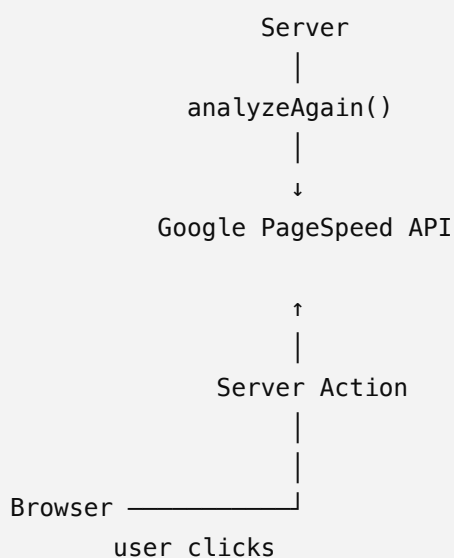
export async function analyzeAgain(url: string) {
  const result = await getPerformanceData(url);

  return result;
}
```



And then use that server function from your UI.

Conceptually:



The important idea is:

Server Actions allow client interactions to invoke functions that execute on the server.

4. So what does "use server" actually mean?

This is the key part.

When you write:

```
</> TypeScript

"use server";

export async function analyzeAgain() {
  // ...
}
```



you're telling Next.js:

This function is server-only code and can be invoked as a Server Action.

It does **NOT** mean:

"Run this entire application on the server forever."

And it does **NOT** mean:

"Make this function magically execute in the browser."

Quite the opposite.

"use server" means:

KEEP THIS CODE ON THE SERVER



That's the mental model I want you to remember.

5. Compare "use client" and "use server"

This is where people often get tangled.

"use client"

```
</> TypeScript

"use client";

function Button() {
```



```
return <button onClick={...}>Click</button>;  
}
```

means:

This module/component needs to participate in the **browser/client environment**.

Therefore it can use things like:

⌄ TypeScript

```
onClick  
useState  
useEffect  
window  
document
```



"use server"

⌄ TypeScript

```
"use server";  
  
export async function saveData() {  
  // server code  
}
```



means:

This function is intended to execute on the **server** and can be invoked remotely from client interactions.

So:

"use client"

↓

Browser/client code

"use server"

↓

Server code / Server Action



6. The really cool part: it looks like a normal function call

Suppose we have:

Server

```
</> TypeScript   
  
"use server";  
  
export async function analyze(url: string) {  
  const result = await fetch(  
    "https://pagespeed.googleapis.com/..."  
  );  
  
  return result.json();  
}
```

And Client Component:

```
</> TypeScript   
  
"use client";  
  
import { analyze } from "../actions";  
  
export function AnalyzeButton() {  
  async function handleClick() {  
    const result = await analyze("example.com");  
  
    console.log(result);  
  }  
  
  return (  
    <button onClick={handleClick}>  
      Analyze  
    </button>  
  );  
}
```

It looks like:

```
handleClick()  
  ↓  
analyze() 
```

as though `analyze()` were a normal local JavaScript function.

But it isn't.

There is actually a **client** → **server communication boundary** there.

Conceptually:

BROWSER



handleClick()

|

invoke Server Action

↓

NETWORK

|

↓

SERVER

analyze()

|

↓

PageSpeed API

|

↓

result

|

↑

NETWORK

|

↓

BROWSER

result

That's the magic.

7. This is why Server Actions exist

Think about our Web Performance Analyzer.

A user enters:

`https://example.com`



and clicks:

Analyze

We need:

User interaction

↓

Server-side mutation/action

↓

Google PageSpeed

↓

possibly Gemini

↓

save/update data

↓

UI updates



Server Actions provide a convenient mechanism for the **user interaction → server operation** part.

Instead of thinking:

"I need to create an API endpoint."

you can think:

"I need a server function that the UI can invoke."

That's a much nicer programming model.

8. One VERY important distinction

Don't confuse:

Server Component

with

Server Action

They're related, but they're **not the same thing**.

Server Component

Used primarily for:

rendering UI

+

fetching data



Example:

```
</> TypeScript   
  
export default async function Page() {  
  const data = await getPerformanceData();  
  
  return <Results data={data} />;  
}
```

The server renders the component.

Server Action

Used primarily for:

user-triggered server operations 

Example:

```
</> TypeScript   
  
"use server";  
  
export async function analyze(url: string) {  
  // perform server-side operation  
}
```

Then:

User clicks
↓
Server Action
↓
server work
↓
UI updates 

So a useful mental model is:

SERVER COMPONENT 
↓
"What should I render?"

SERVER ACTION
↓
"What should I DO when the user interacts?"

🔥 That's the distinction.

9. And this leads directly into our next concepts

Once you understand Server Actions, the rest of Lesson 5 starts making sense:

```
5.1
Why Server Actions?
    ↓
"use server"
    ↓
5.2
Forms + Server Actions
    ↓
5.3
Mutations
    ↓
5.4
revalidatePath / revalidateTag
    ↓
5.5
Optimistic UI
    ↓
5.6
pending/error states
    ↓
5.7
real-world mutation architecture
```



And we'll keep using our **Web Performance Analyzer** as the running example rather than learning this as disconnected syntax.

The one sentence I want locked in:

`"use server"` tells Next.js that the function belongs on the server and can be invoked from a client interaction as a Server Action.

And **it does not mean the browser is running Next.js's server**. The browser still just makes a request to the Next.js server when the action is invoked.

That last distinction connects directly to the confusion you had earlier about **browser vs Next.js BFF vs actual cloud server** — and Server Actions are actually a perfect way to make that architecture click. 🔥



Exactly — **"architectural role"** sounds fancy, but the idea is simple. 😊

🧠 **Architectural role = "what job does this part play?"**

It doesn't necessarily mean a specific piece of code or technology.

For example, in your application:



```
graph TD; Browser --> Nextjs[Next.js]; Nextjs --> GooglePageSpeed[Google PageSpeed];
```

Browser
↓
Next.js
↓
Google PageSpeed



We can say:

Next.js is playing the role of a BFF.

Why?

Because it's sitting between your frontend and backend/external services and doing things like:

- receiving requests from the UI
- calling Google PageSpeed
- keeping API keys/secrets on the server
- combining data from multiple services
- transforming data into what your UI needs

That's the **BFF role**.

BFF doesn't have to be a specific technology

You could build a BFF using:

Next.js
Node.js/Express
Java/Spring
.NET
Python
etc.



The technology changes, but if that server is specifically acting as a **Backend For Frontend**, its **architectural role** is BFF.

Same idea with other terms

"Database" can be a role:

Something responsible for persistent data storage.

It could be:

PostgreSQL
MySQL
MongoDB



"Cache" can be a role:

Something responsible for temporarily storing frequently used data.

It could be:

Redis
Memcached



"BFF" is similarly a role:

A backend designed to serve the needs of a particular frontend.

Now apply that to Route Handlers

A Route Handler is **actual Next.js functionality**:

```
app/api/analyze/route.ts
```



It creates an HTTP endpoint.

BFF is the **architectural purpose** that endpoint may serve.

So:

```
Route Handler  
↓  
"What mechanism?"
```



while:

```
BFF  
↓  
"What architectural job?"
```



That's the distinction. 🔥

One sentence:

A Route Handler is something you implement; a BFF is the role your server-side application is playing.

And importantly, **not every Route Handler is necessarily a BFF**. A webhook Route Handler, for example, is primarily an HTTP entry point for an external service; whether the broader Next.js server is functioning as a BFF depends on the architecture.

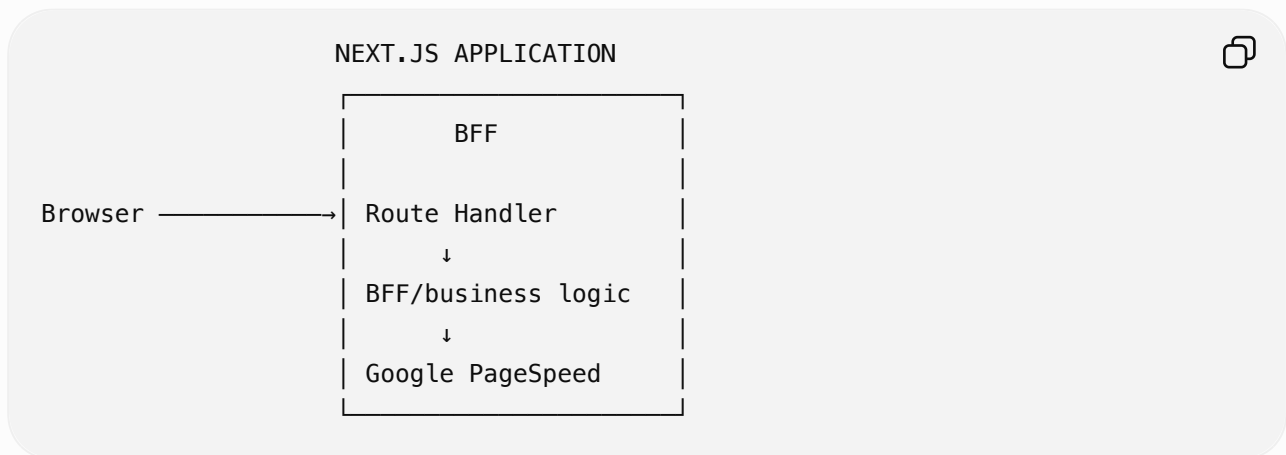


so u r saying route handlers are part of BFF ?

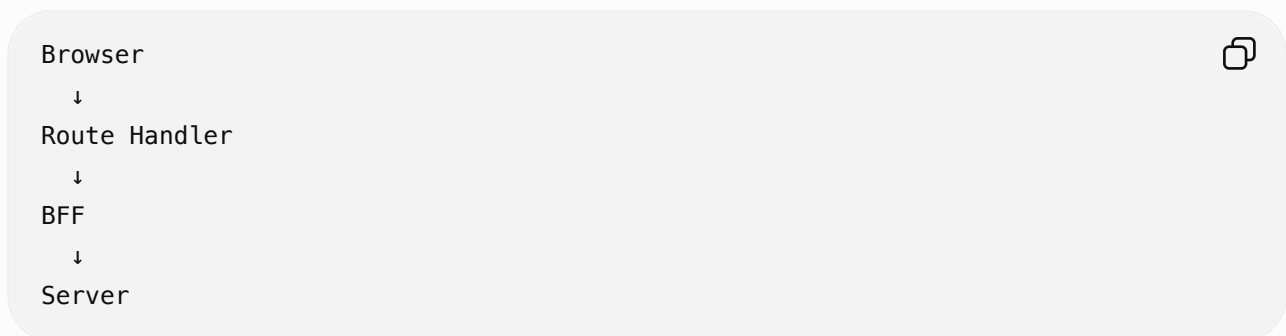
Yes — with one important qualification. 🙌

If your Next.js application is acting as a **BFF**, then its **Route Handlers** can be part of that BFF.

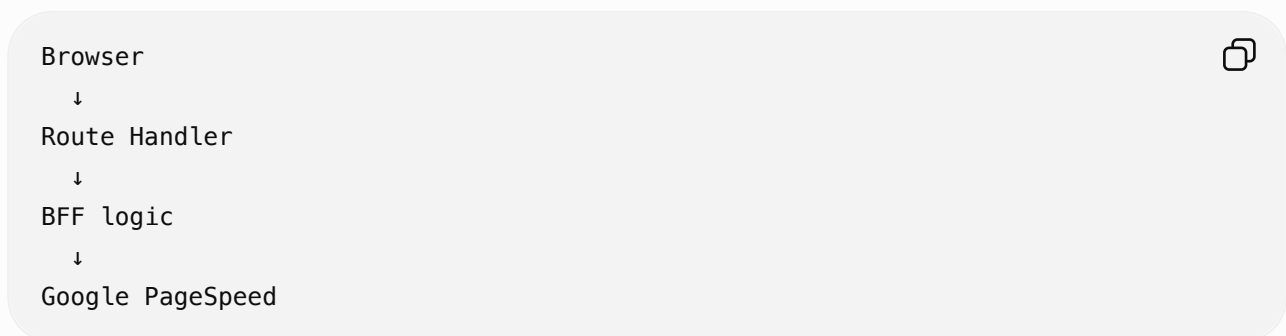
Think:



So **NOT**:



Instead:



And the whole thing is running on the **Next.js server**.

But don't make this rule:

"Every Route Handler = BFF."

That's not true.

A Route Handler is simply an **HTTP endpoint mechanism** in Next.js.

It *can* be used as part of a BFF.

For your project, you could have:

```
/api/analyze
  ↓
Route Handler
  ↓
BFF behavior
  ↓
PageSpeed API
```



while:

```
/api/stripe/webhook
  ↓
Route Handler
  ↓
process webhook
```



is an HTTP endpoint too, but its purpose isn't necessarily BFF behavior.

So the cleanest statement is:

Route Handlers are a Next.js mechanism; when your Next.js server is functioning as a BFF, Route Handlers can be the entry points through which that BFF is accessed. 🧠🔥



okay so route handlers are external doors and server actions are internal doors

YES. 🔥 **That is actually a very good mental model.**

I'd just make one tiny adjustment:

Route Handler = HTTP door

It's an explicit HTTP endpoint that **any appropriate HTTP client** can call:

Browser / Mobile / Stripe / Other service



 Route Handler



Next.js server/BFF

Example:

POST /api/analyze

POST /api/webhook



Server Action = Server-function door

It's primarily a convenient door for **your Next.js UI** to invoke server-side code:

Your Next.js UI



 Server Action



Next.js server/BFF

You don't manually create:

POST /api/analyze



for that interaction.

So your analogy:

Route Handlers = external/public HTTP doors

Server Actions = internal application doors

is **excellent as a learning mental model**.

Just remember that "external" doesn't mean **only external services**. Your browser can absolutely use a Route Handler too.

And "internal" doesn't mean the action literally runs inside the browser—it **executes on the server**.

That's the picture I'd keep in your head going forward. 🙌



ChatGPT can make mistakes. Check important info.